

Loan Default Prediction Using Explainable Gradient Boosting With Cost-Sensitive Learning

Project Report – Round 1 Coding & Data Analysis Task

Submitted by: **Abhijeeta Panigrahi**

1. Introduction

Loan default prediction is an important problem in the financial sector because banks and lending companies often struggle to identify which customers are likely to default on their loans. A wrong decision here can create a huge financial loss. That's why machine learning models are widely used to support credit decision-making.

In this project, I tried to build a model that not only predicts loan default accurately but also focuses more on identifying the minority class (the people who actually default). Usually, this minority class is very small compared to the non-default group, which makes the problem imbalanced.

To improve this, I used **cost-sensitive Gradient Boosting**, which gives more importance to the default class, along with **SMOTE**, which balances the dataset. I also added explainability using feature importance so we can understand how the model makes decisions.

My goal was to check whether this improved approach performs better than traditional models like Logistic Regression and Random Forest.

2. Literature Review

For this project, I went through several research papers from IEEE, Springer, and other reliable sources. Most papers focused on applying classic machine learning algorithms such as Logistic Regression, Decision Trees, Random Forest, and sometimes ANN-based deep learning.

Paper 1 – Logistic Regression for Credit Scoring

This paper used Logistic Regression to classify credit customers. Although it worked decently, the model struggled with imbalanced datasets. Logistic Regression also doesn't capture non-linear patterns well.

Paper 2 – Random Forest for Loan Default Classification

Random Forest performed better than Logistic Regression because it handles complex relationships

and interactions. However, the paper mentioned that interpretability becomes difficult and the model still didn't perform well on minority classes.

Paper 3 – Deep Learning for Credit Risk Detection

This paper used neural networks. The accuracy was high, but the computational cost was also high. Also, the black-box nature of deep learning makes it difficult to explain why the model predicted default for a customer.

Research Gaps Identified

After reading the papers, I noticed a few common issues:

- Most models didn't handle class imbalance properly.
- Default class (1) is usually very small, but misclassifying it has huge consequences.
- Many papers didn't use cost-sensitive techniques.
- Very few models included explainability, making it hard for financial institutions to trust the results.

3. Research Questions

I decided to focus on the following main questions:

1. Can cost-sensitive Gradient Boosting improve the detection of loan defaulters compared to regular baseline models?
2. Does applying SMOTE or class weights help improve recall for the minority class?
3. Can feature importance or SHAP add explainability to the model's decision-making?

4. Objectives

- Build baseline models (Logistic Regression and Random Forest) for comparison.
- Implement a cost-sensitive Gradient Boosting model (XGBoost).
- Handle class imbalance using SMOTE and `scale_pos_weight`.
- Compare the metrics (Accuracy, Recall, F1-score, and AUC).
- Add explainability so the model is not a black box.
- Prepare visual results like ROC curves, confusion matrices, and feature importance plots.

5. Proposed Methodology

5.1 Pipeline Architecture

Here is the flow I followed in the project:

1. Load the dataset
2. Clean missing values
3. Encode categorical data
4. Split into training and testing datasets
5. Apply SMOTE to balance the training set
6. Train baseline models
7. Train the main gradient boosting model
8. Evaluate all models
9. Plot important graphs
10. Compare results and conclude

5.2 Algorithm (Written in My Own Words)

Step 1: Read the dataset and separate the target column.

Step 2: Clean the data (missing values, encoding, etc.).

Step 3: Split the data into training and test sets.

Step 4: Apply SMOTE on the training data to fix imbalance.

Step 5: Train Logistic Regression and Random Forest models.

Step 6: Calculate class weights and train the proposed XGBoost model.

Step 7: Test all models and compute performance metrics.

Step 8: Generate ROC curves and confusion matrices.

Step 9: Extract feature importance for explainability.

Step 10: Compare all models and decide which one works best.

6. Data Handling & Preprocessing

The preprocessing was done using a clean pipeline approach:

- Missing numeric values → replaced with median
- Missing categorical values → replaced with most frequent
- Categorical variables → OneHotEncoding
- Splitting → 80% training, 20% testing
- Imbalance handling → SMOTE + scale_pos_weight for XGBoost

In the final PDF I also include screenshots of:

- First rows of the dataset
- Missing value summary
- Class distribution before and after SMOTE.

7. Model Development

Baseline Models

1. Logistic Regression – simple and fast, but often underperforms on imbalanced data.
2. Random Forest – usually stronger performance but still not fully optimized for the minority class.

Proposed Model: Gradient Boosting (XGBoost)

This algorithm builds trees step-by-step and focuses more on difficult samples. The main improvement I introduced was using:

- scale_pos_weight → gives more weight to defaulters
- Balanced training set via SMOTE

This combination works well for loan prediction tasks.

8. Key Code Snippets

Below are some important parts of the code I wrote for this project. The full codebase is organized into preprocessing, models, evaluation, and main execution files.

```
# preprocessing.py (simplified)

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

def load_dataset(file_path: str) -> pd.DataFrame:
    data = pd.read_csv(file_path)
    return data

def prepare_data(df: pd.DataFrame, target_col: str):
    X = df.drop(columns=[target_col])
    y = df[target_col]

    numeric_cols = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
    categorical_cols = X.select_dtypes(include=["object", "bool"]).columns.tolist()

    numeric_pipe = Pipeline(steps=[("fill_num", SimpleImputer(strategy="median"))])
    categorical_pipe = Pipeline(steps=[
        ("fill_cat", SimpleImputer(strategy="most_frequent")),
        ("encode", OneHotEncoder(handle_unknown="ignore")),
    ])

    preprocessor = ColumnTransformer(
        transformers=[
            ("num_process", numeric_pipe, numeric_cols),
            ("cat_process", categorical_pipe, categorical_cols),
        ]
    )

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=15, stratify=y
    )

    return preprocessor, X_train, X_test, y_train, y_test

# models.py (simplified)

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

def get_comparison_models(apply_class_weight=True):
    weight = "balanced" if apply_class_weight else None

    lr_model = LogisticRegression(max_iter=1000, n_jobs=-1, class_weight=weight)
    rf_model = RandomForestClassifier(
        n_estimators=160, random_state=15, class_weight=weight, n_jobs=-1
    )

    return {
        "Logistic Regression": lr_model,
        "Random Forest": rf_model,
    }

def build_main_model(pos_weight: float = 1.0):
    gb = XGBClassifier(
        n_estimators=260,
        learning_rate=0.05,
        max_depth=4,
```

```

        subsample=0.85,
        colsample_bytree=0.85,
        objective="binary:logistic",
        scale_pos_weight=pos_weight,
        random_state=15,
        n_jobs=-1,
        eval_metric="logloss",
    )
    return gb

# main.py (simplified)

from imblearn.over_sampling import SMOTE
from preprocessing import load_dataset, prepare_data
from models import get_comparison_models, build_main_model
from evaluation import calculate_metrics, show_confusion_matrix, plot_roc_for_models

def main():
    df = load_dataset("data/loan_data.csv")
    target = "default"
    preprocessor, X_train, X_test, y_train, y_test = prepare_data(df, target)

    X_train_processed = preprocessor.fit_transform(X_train)
    X_test_processed = preprocessor.transform(X_test)

    smote = SMOTE(random_state=15)
    X_train_bal, y_train_bal = smote.fit_resample(X_train_processed, y_train)

    pos = (y_train == 1).sum()
    neg = (y_train == 0).sum()
    pos_weight = neg / pos if pos > 0 else 1

    baseline_models = get_comparison_models(apply_class_weight=True)
    # train and evaluate baseline models...
    gb_model = build_main_model(pos_weight=pos_weight)
    # train and evaluate proposed model...

```

9. Visualizations

In my full project, I generate and save several visualizations using matplotlib:

- ROC Curve comparison for all models
- Confusion matrices
- Feature importance bar chart

These plots are then added as screenshots into the report and help explain the performance of each model visually.

10. Comparative Analysis

After running all the models, I compare them based on Accuracy, Precision, Recall, F1-score, and AUC. In general, the proposed Gradient Boosting model with cost-sensitive learning and SMOTE performs better, especially in terms of Recall and AUC for the default class.

This makes it a more reliable model for identifying risky customers compared to the baseline approaches.

11. Conclusion and Future Work

From this project, I learned how important it is to handle imbalance when dealing with real-world financial datasets. My proposed model performed better than the baseline models in most metrics, especially recall and AUC, which are crucial for identifying potential defaulters.

Using explainability tools also added transparency, which is necessary in banking scenarios. Overall, the combination of SMOTE plus cost-sensitive Gradient Boosting provided a strong improvement over traditional approaches.

In the future, I would like to:

- Perform more extensive hyperparameter tuning.
- Add SHAP-based explainability.
- Try other boosting algorithms like LightGBM or CatBoost.
- Experiment with additional features or alternative datasets.